

Hibon Sacha

22 avril 2025

LANGUAGE

-C

GROUPE A:
CHAMPION
DES MATHS



SOMMAIRE

DE LA PRÉSENTATION

Présenté par Hibon Sacha

01

Introduction

02

Fonctionnalités implémentées

03

Difficultés rencontrées et solutions

04

Commentaires et suggestions

05

Auto-évaluation

06

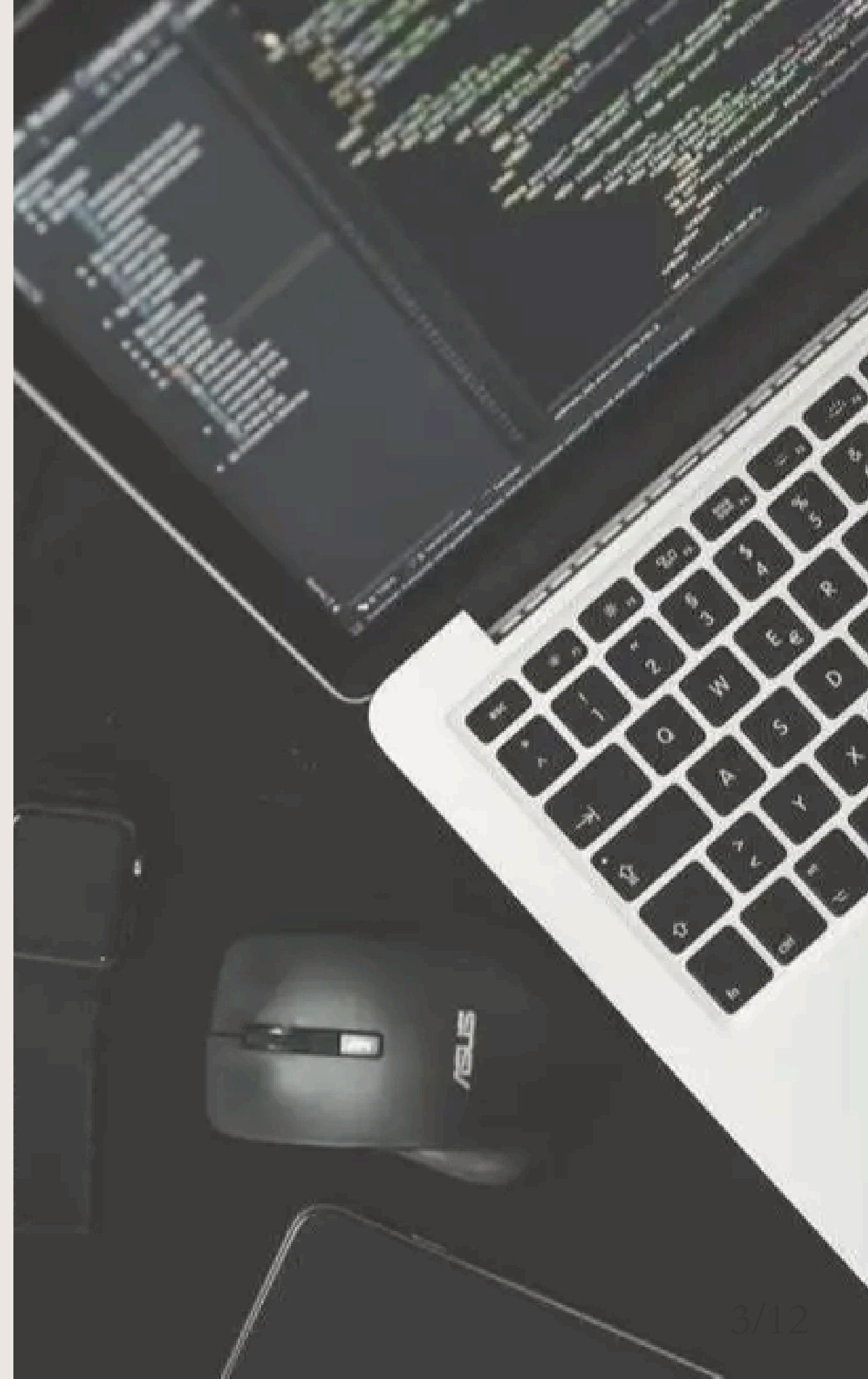
Structure du dépôt et code

INTRODUCTION

Ce mini-projet en langage C a été réalisé dans le but de créer un jeu éducatif sur les mathématiques. L'idée est d'aider des élèves de CM1 à s'entraîner de manière amusante avec des calculs : additions, soustractions, multiplications, divisions, mais aussi des exercices plus complexes comme écrire un grand nombre en chiffres ou convertir des heures en minutes.

Le programme fonctionne en ligne de commande. Il affiche un menu, propose des exercices, note le joueur et garde son score en mémoire grâce à un fichier texte. C'est un projet qui m'a permis de mieux comprendre comment structurer un programme en C avec plusieurs fonctions et comment lire/écrire dans un fichier.

Présenté par Hibon Sacha



FONCTIONNALITES IMPLEMENTEES

Quand on lance le programme, on voit un menu comme celui-ci

```
: Entrez votre nom : sarah
Votre score actuel est : 0
+-----+
|1 : Addition      |
|2 : Soustraction  |
|3 : Multiplication|
|4 : Tables des multiplications|
|5 : Division      |
|6 : exerciceTableMultiplication|
|7 : Ecriture des grands nombres|
|8 : Jeu conversion de duree    |
|0 : Sortir du jeu  |
+-----+
Quel est votre choix ? █
```

L'utilisateur choisit un jeu en entrant le numéro correspondant. Ce menu est affiche en boucle tant que l'utilisateur ne tape 0. Chaque chiffre appelle une fonction différente avec switch(choix).

Les jeux disponibles sont Addition / Soustraction / Multiplication / Division/Conversion/Ecriture.

Chaque opération propose :

- des nombres générés au hasard,
- trois tentatives pour trouver la bonne réponse,
- un système de points (10 points au 1er essai, 5 au 2e, 1 au 3e),
- la bonne réponse affichée si on ne trouve pas.

Par exemple la fonction addition() va générer au hasard deux nombres avec rand() % .L'utilisateur a 3 essais le système de pointage est réalisé grâce à un do{ ...}while (essais < 3) essais ici est un entier initialisé à 0 et à l'intérieur de ce while nous avons plusieurs if imbriqués.

```
Quel est votre choix ? 1

Combien fait 10 + 57 ? 6
Mauvaise réponse. Réessayez.

Combien fait 10 + 57 ? 4
Mauvaise réponse. Réessayez.

Combien fait 10 + 57 ? 67
Correct mais tard ! +1 point
Score sauvegarder avec succes
```

De la même manière on réalise les autres fonctions.

Pour le jeu nombre en lettre deux autres fonctions ont du etre réalisées .Tout d'abord la fonction convertir_certaine qi traite les centaines,dizaines etc,par exemple pour les centaines on aura :

Elle va donc faire une division par cent,ce qui permet donc d'extraire la valeur des centaines.Par exemple, pour n = 345, centaines = 3. Si le nombre est exactement 100 (par exemple, n = 100), il ajoute simplement "cent".Et enfin Utilisation de strcat pour ajouter la représentation textuelle des centaines. Exemple : pour n = 345, cette partie ajouterait "trois cent". Cela fonctionne de la meme maniere pour les unités etc.

```
if (n >= 100) {
    int centaines = n / 100;
    if (centaines == 1)
        strcat(buffer, "cent");
    else {
        strcat(buffer, unites[centaines]);
        strcat(buffer, " cent");
    }
    if (n % 100 != 0)
        strcat(buffer, " ");
}
```

La fonction nombre_en_lettres traite les millions et millier par exmple pour les millions on aura une division par 1 000 000 qui le nombre de millions. Exemple : pour n = 1234567, millions = 1.Pour le cas "un million" : Traite le cas spécifique où le nombre est exactement 1 million.Pour la concaténation il ajoute "millions" à la chaîne finale.Ainsi dans la fonction jeu_nombre_en_lettres génère un nombre aléatoire entre 1 000 000 et 9 000 000 grâce à rand()%.Et utilise la fonction nombre_en_lettres pour transformer le nombre en sa version textuelle et affiche la version textuelle à l'utilisateur.

```
nombre_en_lettres(nombre, lettres);
printf("\nECRITURE DE GRANDS NOMBRES - CM1\n");
printf("Écris ce nombre en chiffres :\n%s\n", lettres);
```

Pour le jeu convertir_duree on aura :

```
int heures = rand() % 10 + 1;    // 1 à 10 h
int minutes = rand() % 60;       // 0 à 59 min
int bonne_reponse = heures * 60 + minutes;
```

Cette fonction va dans un premier temps générer des valeur au hasard avec rand() pour chaque partie (heures et minutes). Puis elle va convertir les heures en minutes en multipliant par 60, puis en ajoutant les minutes restantes.

- Ensuite concernant l'affichage des informations du dernier score enregistré pour un joueur spécifique, comme le score, la date et l'heure. Plusieurs fonctions ont été réalisées notamment `AfficherDernierScoreJoueur` qui donc ouvre le fichier `FICHIER_SCORES` et si le fichier n'existe pas, un message est affiché et la fonction s'arrête.

```
FILE *f = fopen(FICHIER_SCORES, "r");

if (f == NULL) {

    printf("Aucun score enregistré pour le moment.\n");

    return;
```

- Après il procède à la lecture du fichier `fscanf` pour lire les informations du fichier ligne par ligne (nom, date, heure, score). La fonction `strcmp` compare le nom du joueur recherché avec celui dans le fichier. Et si une correspondance est trouvée, les données sont sauvegardées (score, date, heure).

```
while (fscanf(f, "%s %s %s %d", nomFichier, date, heure, &scoreFichier) == 4) {

    if (strcmp(nom, nomFichier) == 0) {

        dernierScore = scoreFichier;

        strcpy(derniereDate, date);

        strcpy(derniereHeure, heure);
```

- Après il procède à la lecture du fichier `fscanf` pour lire les informations du fichier ligne par ligne (nom, date, heure, score). La fonction `strcmp` compare le nom du joueur recherché avec celui dans le fichier. Et si une correspondance est trouvée, les données sont sauvegardées (score, date, heure). Ainsi les informations du dernier score sont affichées dans un format clair.

```
Dernier score enregistré de sacha : 5 (Date : 14/04/2025, Heure : 10:01:22)
```

- Ensuite nous avons la fonction `ChargerScore` qui charge et retourne le dernier score enregistré pour un joueur spécifique. Elle parcourt des lignes du fichier avec `fscanf`. De plus la fonction compare les noms avec `strcmp` pour identifier le score du joueur correspondant

```
while (fscanf(f, "%s %s %s %d", nomFichier, date, heure, &scoreFichier) == 4) {

    if (strcmp(nom, nomFichier) == 0) {

        dernierScore = scoreFichier;
```

- Et enfin elle retourne le score le plus récent trouvé pour le joueur.

```
sacha 14/04/2025 10:01:22 5
```

DIFFICULTES RENCONTREES ET SOLUTIONS

PROBLÈMES RENCONTRÉS

- Les nombres générés étaient toujours les mêmes
- Difficile de récupérer le bon score dans le fichier
- Écrire un nombre en lettres était compliqué
- Parfois le score ne s'enregistrait pas
- Manque de temps

SOLUTIONS

- J'ai utilisé `srand(time(NULL))`; au début du programme pour rendre les tirages vraiment aléatoires
- J'ai comparé le nom de l'utilisateur à chaque ligne pour garder le dernier score trouvé
- J'ai créé des fonctions séparées pour gérer les centaines, les milliers et les millions
- J'ai fait en sorte que le fichier soit mis à jour juste après chaque bonne réponse
- Continuer le travail chez soi

COMMENTAIRES ET SUGGESTIONS

Ce TP m’a permis de mieux maîtriser plusieurs concepts clés en programmation :

- La structuration d’un programme en fonctions modulaires, ce qui rend le code plus lisible et plus facile à maintenir.
- La gestion des variables et des opérateurs pour effectuer des calculs, comparer des valeurs, et construire des conditions logiques.
- L’utilisation des blocs conditionnels imbriqués (par exemple pour attribuer un score selon le nombre d’essais).
- J’ai mieux compris comment lire et écrire des données dans des fichiers en utilisant des fonctions comme `fopen`, `fscanf` et `fclose`. Ces techniques permettent de gérer efficacement des données persistantes, comme des scores enregistrés.
- J’ai renforcé ma maîtrise des fonctions essentielles en C, telles que `strcmp`, `strcpy`, `strlen`, et `sprintf`, pour manipuler et formater dynamiquement des chaînes. Cela m’a permis de gérer des affichages complexes comme la construction de phrases pour les scores ou les conversions.

Sur le plan méthodologique

- J’ai su modéliser chaque fonction indépendamment et tester chaque bloc avant de passer à l’étape suivante.
- L’approche de type “tester au fur et à mesure” a été très efficace pour corriger rapidement les erreurs.

Améliorations:

Pour améliorer le TP, il serait intéressant d’enrichir les consignes initiales avec un exemple complet attendu pour chaque exercice.

Applications futures:

Et pour finir ces apprentissages me seront utiles dans de futurs projets, notamment pour construire des interfaces plus interactives, structurer un projet plus grand ou collaborer sur du code partagé.

ÉLÉMENT	NOTE /5	COMMENTAIRE
Organisation du code	4	J'ai séparé les jeux dans des fonctions claires
Fonctionnement du jeu	5	Tout marche correctement, avec les points et les essais
Tests	1.5	J'ai testé à la main mais pas avec des outils
Documentation	2	Je n'ai pas tout commenté dans le code
Autonomie	2	J'ai fait avec l'aide d'une collegue et quelquefois celle du professeur lorsque c'était trop compliqué.

Ce que je vais travailler à l'avenir :

- Mieux commenter mon code.
- Lire plus sur les structures (comme les tableaux et pointeurs).

STRUCTURE DU DEPOT ET CODE